Day 8
Salesforce Asynchronous Apex

Future Methods · Queueable Apex · Batch Apex · Scheduled Apex

---

Project: Placement Management System
Org: vishnuinstituteoftechnol68 (Developer Edition)
Developer: Sudharshan Kuppili
Documentation Type: Learning & Hands-On Implementation Record

Day 8 Overview

Day 8 focused on Asynchronous Apex on the Salesforce platform — the set of tools that allow work to be moved out of the immediate user transaction and executed in the background. The day combined conceptual learning (synchronous vs. asynchronous processing, and the four major asynchronous mechanisms) with hands-on implementation inside a Developer Edition org, applied to the Placement Management System project.

Four asynchronous Apex types were studied and implemented: Future Methods, Queueable Apex, Batch Apex, and Scheduled Apex. Each was connected back to the **Placement_Application__c** and **Student__c** objects already used elsewhere in the project.

Learning Objectives

- Understand the difference between synchronous and asynchronous Apex execution.
- Recognize when background processing is appropriate for a business operation.
- Understand Future Methods and their role in simple background work.
- Understand Queueable Apex and its use for flexible, job-oriented background work.
- Understand Batch Apex and how it processes large volumes of records in manageable chunks.
- Understand Scheduled Apex and how it triggers work at a specific time.
- Understand that all asynchronous Apex remains subject to Governor Limits and must stay bulk-safe.
- Apply each asynchronous type to a small, real implementation in the Placement Management System.

Concepts Learned

Synchronous vs. Asynchronous Apex: Synchronous Apex executes as part of the user's immediate transaction, so the user waits for it to finish. Asynchronous Apex is handed off to Salesforce to execute separately, so the user does not need to wait for it to complete.

Governor Limits still apply: Asynchronous execution contexts have their own Governor Limits. Moving code to the background does not remove the need for bulk-safe design — SOQL and DML operations still need to be written to handle multiple records safely.

Choosing the right tool: Future Apex suits simple, one-off background work; Queueable Apex suits flexible background work, including job chaining; Batch Apex suits processing large volumes of records in manageable chunks; and Scheduled Apex suits work that must run at a specific time or on a recurring schedule. Scheduled Apex can also be combined with Batch Apex for scheduled large-volume processing.

1. Future Apex

Explanation

A Future Method is the simplest form of asynchronous Apex. It is declared with the `@future` annotation, must be `static` and return `void`, and its parameters are restricted to primitive types, collections of primitives, and similar simple types (it cannot accept sObjects directly). `@future(callout=true)` is used specifically when the method needs to make an external callout.

When to Use It

Future Methods are appropriate for simple, non-urgent background work — for example, processing a single record's follow-up logic without making the user wait for it.

Implementation Completed — PlacementFutureService

A class named `PlacementFutureService` was created with a static `@future` method, `processApplication`, that receives a Placement Application record Id and queries the corresponding `Placement_Application__c` record inside the asynchronous transaction, then logs a debug statement confirming the record processed.

The method was tested using Execute Anonymous against test record `a0ANS00000CUXcL2AX`, and the execution completed successfully with the expected debug output.

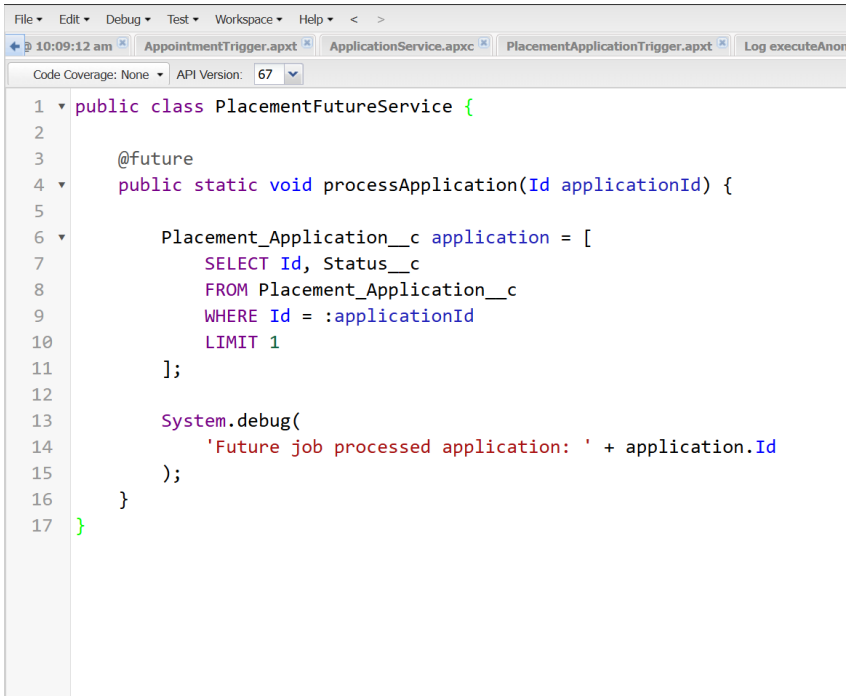

Figure 1: Future Apex implementation — PlacementFutureService.processApplication()

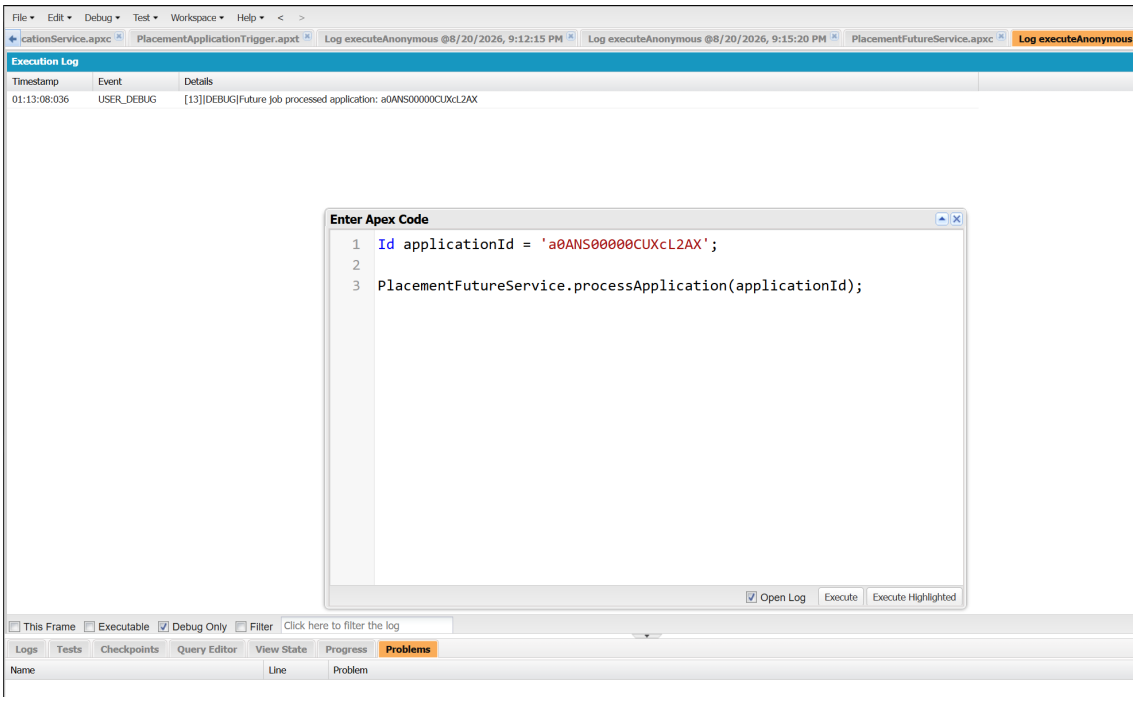

Figure 2: Successful Future Apex execution — debug log confirming "Future job processed application: a0ANS00000CUXcL2AX"

2. Queueable Apex

Explanation

Queueable Apex offers a more flexible, job-oriented model for background processing than Future Methods. A class implements the `Queueable` interface and defines an `execute(QueueableContext context)` method. Unlike Future Methods, Queueable classes can accept complex parameters (including sObjects) through a constructor, and jobs can be chained by enqueuing a new job from within `execute()`.

When to Use It

Queueable Apex is suited to background work that needs more structure than a Future Method, including passing in specific state through a constructor or chaining a sequence of jobs.

Implementation Completed — PlacementQueueableService

A class named `PlacementQueueableService` was created implementing `Queueable`. It receives a Placement Application Id through its constructor, and its `execute(QueueableContext context)` method queries the corresponding `Placement_Application__c` record and logs a debug statement confirming the record processed. The job was submitted using `System.enqueueJob()`.

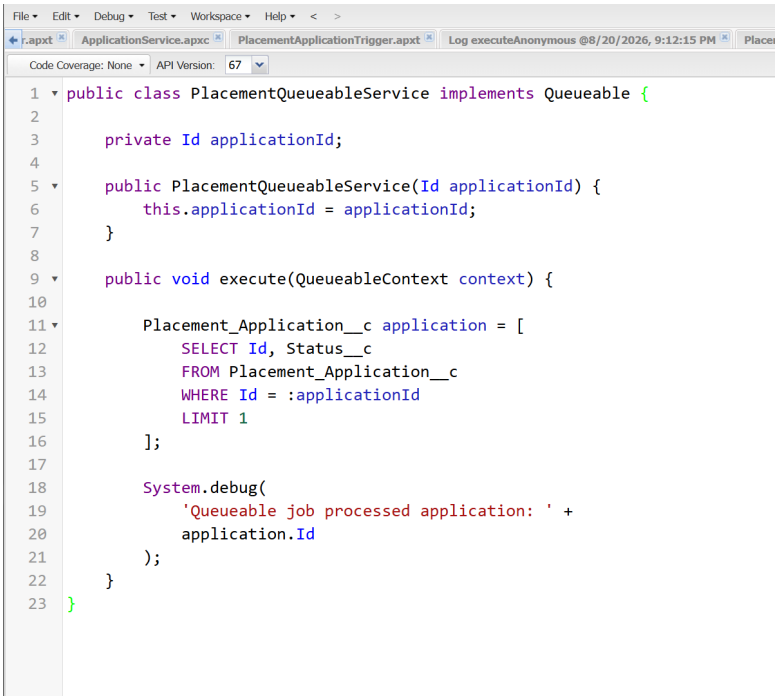

Figure 3: Queueable Apex implementation — PlacementQueueableService

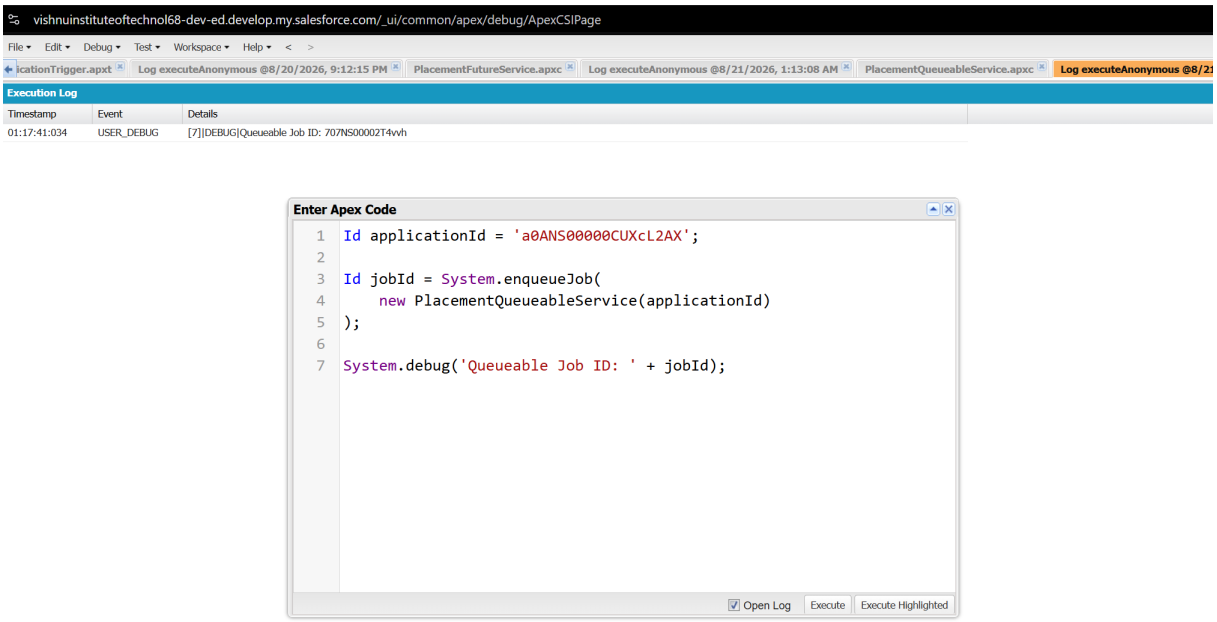

Figure 4: Queueable job submitted via System.enqueueJob() — Job ID 707NS00002T4vvh logged to the debug output

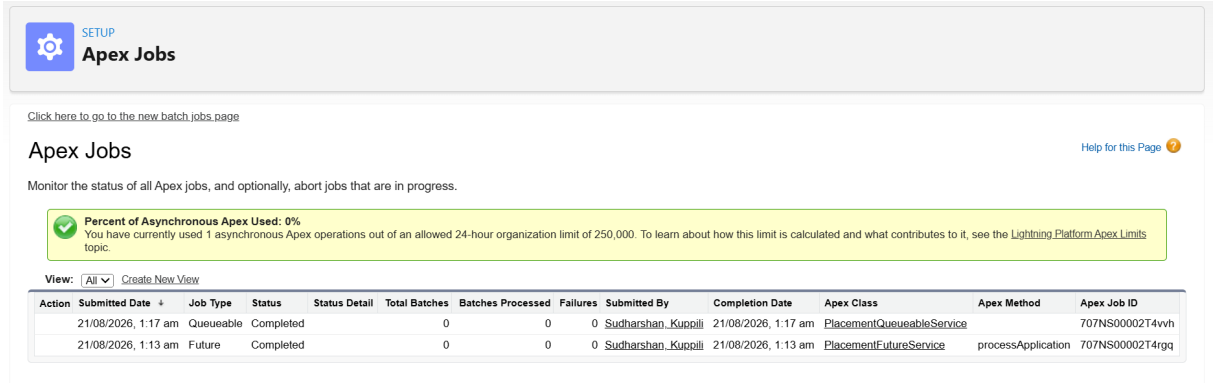

Figure 5: Apex Jobs list in Setup showing PlacementQueueableService and PlacementFutureService both Completed with 0 Failures

3. Batch Apex

Explanation

Batch Apex is designed for processing large volumes of records by dividing them into manageable chunks. A class implements `Database.Batchable<SObject>` and defines three methods:

- `start(Database.BatchableContext bc)` — returns a `Database.QueryLocator` identifying the records to process.
- `execute(Database.BatchableContext bc, List<SObject> scope)` — processes one batch (chunk) of records at a time.
- `finish(Database.BatchableContext bc)` — runs once, after all batches have completed.

When to Use It

Batch Apex is appropriate when a very large number of records needs to be processed and the operation would exceed the limits of a single transaction if run all at once.

Implementation Completed — PlacementStudentBatch

A class named `PlacementStudentBatch` was created implementing `Database.Batchable<SObject>`. Its `start()` method retrieves `Student__c` records using `Database.getQueryLocator()`, `execute()` logs a debug statement for each student in the batch, and `finish()` logs a completion message. The job was started using `Database.executeBatch(new PlacementStudentBatch(), 200)` with a batch size of 200, and it executed and completed successfully.

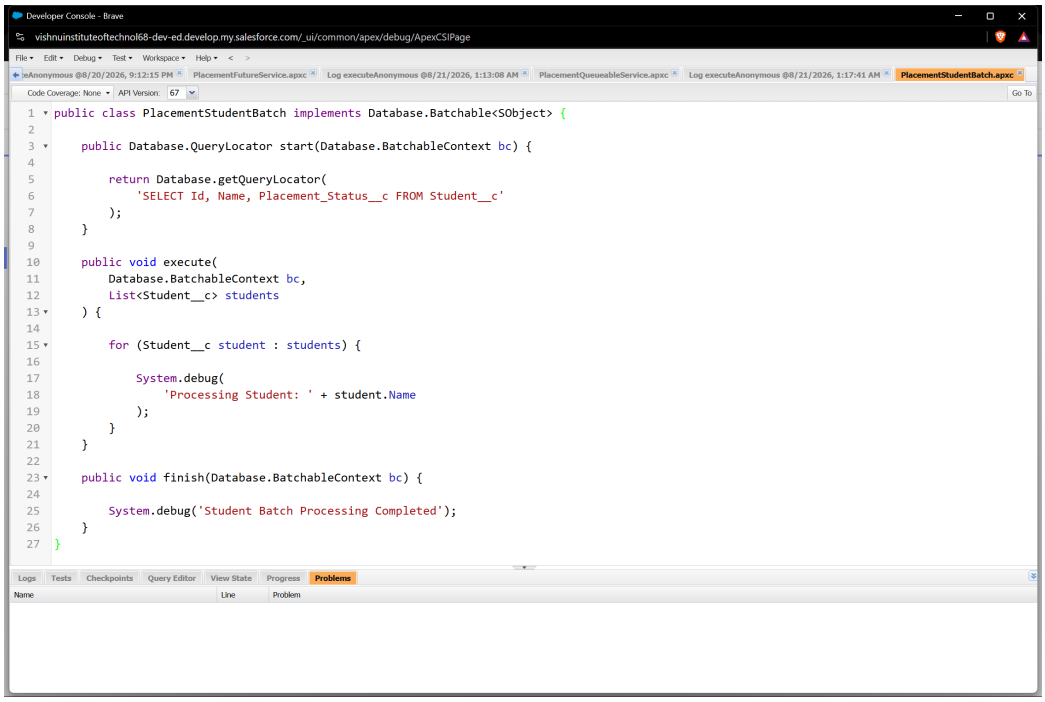

Figure 6: Batch Apex implementation — PlacementStudentBatch (start / execute / finish)

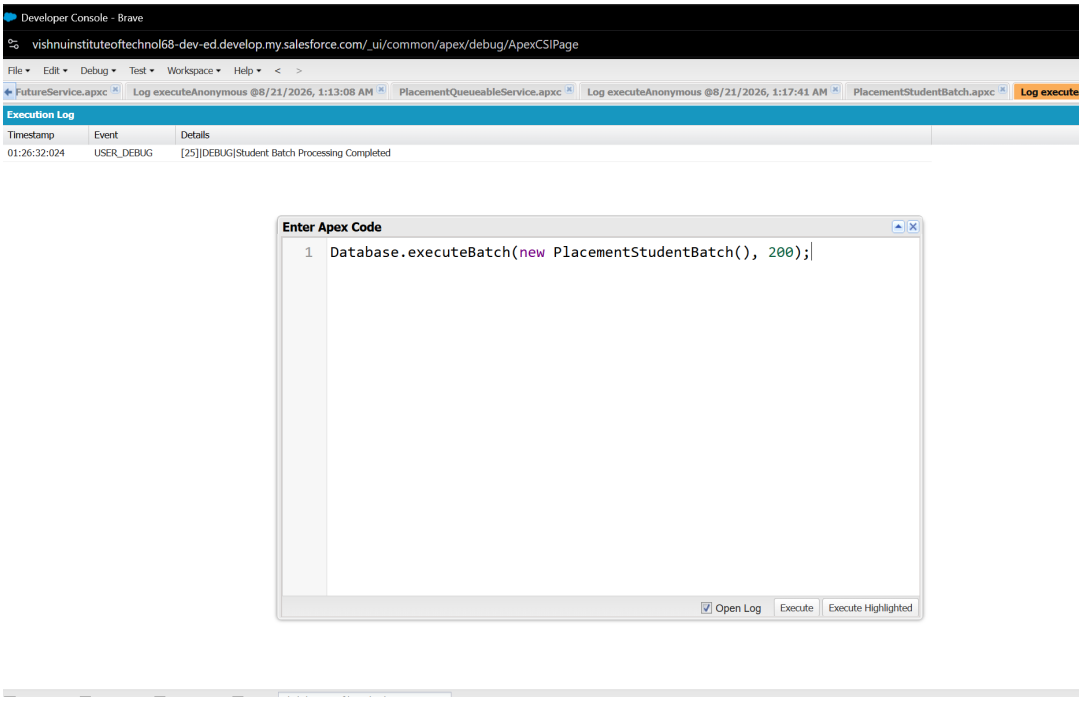

Figure 7: Batch Apex started via Database.executeBatch(new PlacementStudentBatch(), 200) — debug log confirms "Student Batch Processing Completed"

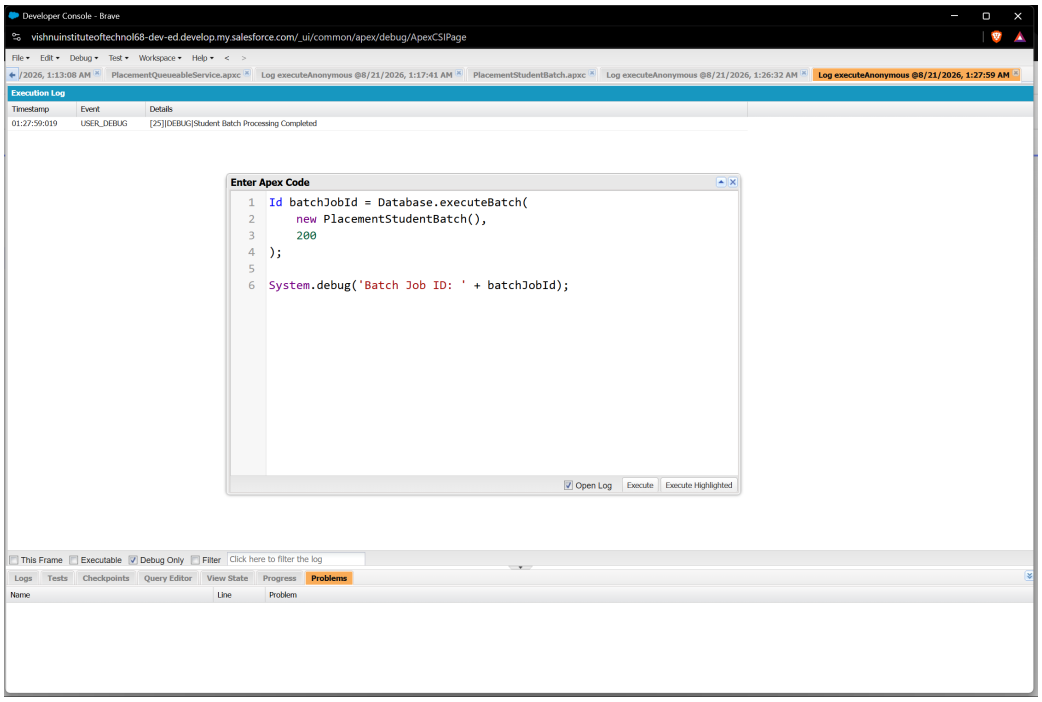

Figure 8: Batch job re-run capturing the returned Batch Job ID via debug output, confirming successful completion

4. Scheduled Apex

Explanation

Scheduled Apex allows code to run at a specific time or on a recurring schedule. A class implements the `Schedulable` interface and defines an `execute(SchedulableContext sc)` method. The job is scheduled using `System.schedule()`, which takes a job name, a CRON expression, and an instance of the Schedulable class.

When to Use It

Scheduled Apex is appropriate when a business process needs to run automatically at a specific or recurring time, rather than in response to a user action. When the scheduled process also involves a very large number of records, Scheduled Apex is commonly used to start a Batch Apex job at the scheduled time.

Implementation Completed — PlacementScheduledService

A class named `PlacementScheduledService` was created implementing `Schedulable`. Its `execute(SchedulableContext sc)` method queries a limited set of `Student__c` records and logs debug statements marking the start, per-student checks, and completion of the run.

The job was scheduled using `System.schedule()` with the job name **"Daily Placement Maintenance"** and the CRON expression `0 0 22 * * ?`, which represents a daily 10:00 PM schedule. The scheduling call executed successfully and returned a Scheduled Job ID.

*The screenshots below confirm that the job was successfully **created and scheduled**. They do not show the 10:00 PM execution itself, so no claim is made that the scheduled run has yet occurred or been observed.*

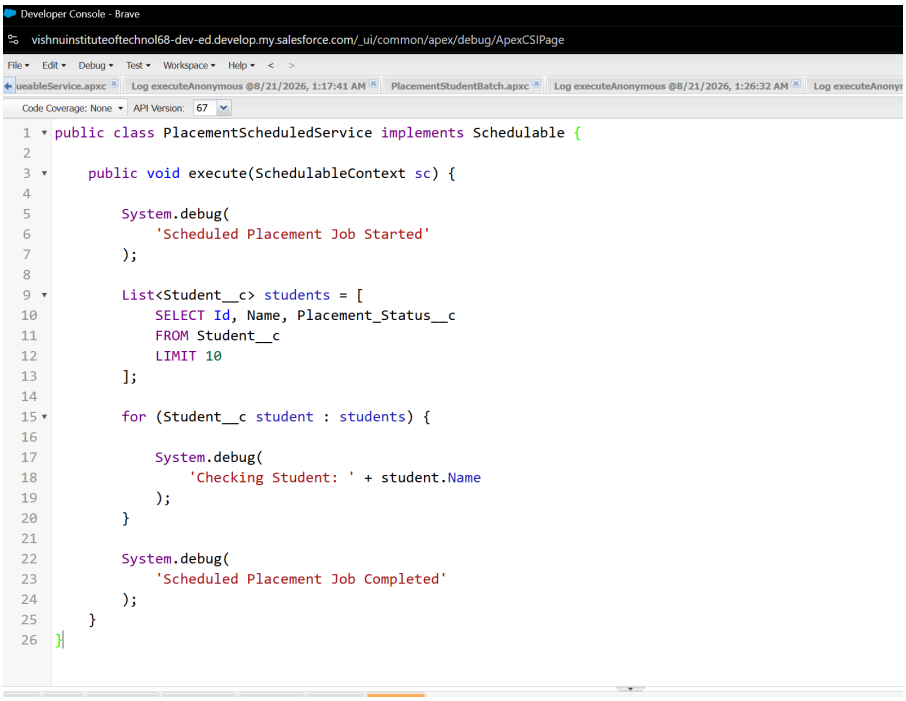

Figure 9: Scheduled Apex implementation — PlacementScheduledService

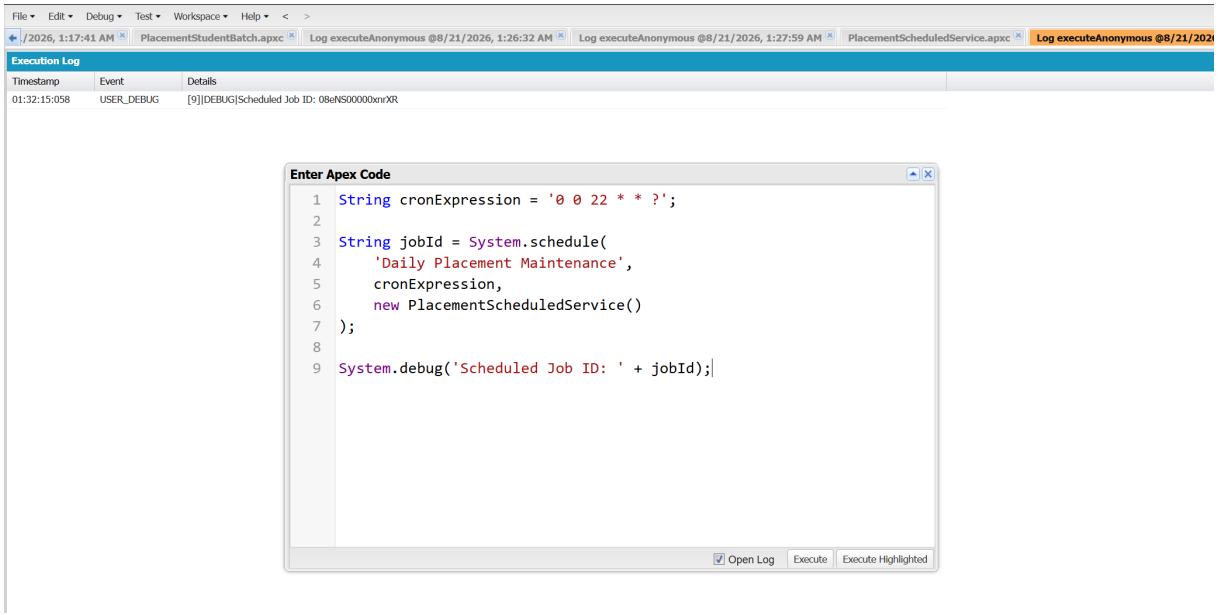

*Figure 10: Scheduled job created via System.schedule("Daily Placement Maintenance", "0 0 22 * * ?", new PlacementScheduledService()) — Scheduled Job ID 08eNS00000xnrXR logged to the debug output*

5. Comparison Table

Type	Main Purpose	Key Method / API	Best Use Case
Future	Simple background work	`@future`	Simple asynchronous processing
Queueable	Flexible / chained background work	`System.enqueueJob()`	Flexible async processing, job chaining
Batch	Large-volume processing	`Database.executeBatch()`	Large datasets processed in chunks
Scheduled	Time-based execution	`System.schedule()`	Scheduled / recurring jobs

6. Hands-on Implementation Summary

Type	Class Created	Verified Result
Future	`PlacementFutureService`	Executed successfully; debug output verified
Queueable	`PlacementQueueableService`	Executed successfully; Apex Jobs list shows Completed, 0 Failures
Batch	`PlacementStudentBatch`	Executed successfully with batch size 200; debug output verified
Scheduled	`PlacementScheduledService`	Successfully scheduled ("Daily Placement Maintenance", 10:00 PM); execution not yet observed

All four classes were built for the Placement Management System and reference `Placement_Application__c` or `Student__c` records. Future, Queueable, and Batch executions were run and verified in the Developer Console and, for Queueable and Future, cross-checked in the Setup Apex Jobs list. The Scheduled Apex job was created and its Job ID confirmed via debug log; its 10:00 PM run was not directly observed during this session.

7. Practical Scenarios

As part of Day 8, a scenario-based review was completed to reinforce when each asynchronous type applies:

- **Simple, non-urgent background operation** → Future Apex
- **Flexible background operation / chaining required** → Queueable Apex
- **Very large data volume** → Batch Apex
- **Work must run at a specific or recurring time** → Scheduled Apex
- **Scheduled, large-volume processing** → Scheduled Apex triggering Batch Apex

8. Key Takeaways

- Not all work needs to happen inside the user's immediate transaction — asynchronous Apex lets secondary work move to the background.
- Future Apex is best for simple, one-off background tasks with primitive parameters.
- Queueable Apex offers a more structured, flexible model and supports job chaining.
- Batch Apex is built specifically for processing large volumes of records in safe, manageable chunks.
- Scheduled Apex triggers work based on time, and can be combined with Batch Apex for large scheduled jobs.
- Asynchronous Apex still has Governor Limits — bulk-safe design remains essential regardless of execution context.
- Verifying results (debug logs, Apex Jobs) is as important as writing the asynchronous code itself.

Conclusion

Day 8 covered the four core mechanisms of Asynchronous Apex on the Salesforce platform — Future Methods, Queueable Apex, Batch Apex, and Scheduled Apex — along with the reasoning for choosing one over another based on the nature of the work. All four types were implemented as small, focused classes tied to the Placement Management System: Future, Queueable, and Batch Apex were executed and their results verified in Salesforce, while Scheduled Apex was successfully configured and confirmed as scheduled. This gives the project a working foundation in background processing that can be extended as the Placement Management System grows.